Efficient Differentially Private Cardinality Estimation for Sensitive Datasets

Thuc X. Nguyen

February 26TH, 2024

Table of Contents

Abstract.....3
1. Introduction.....4
2. Background..... 4
2.1 Counting with The Power Law..... 4
2.2 LogLog Algorithm..... 5
2.3 HyperLogLog Algorithm..... 6
2.4 Differential Privacy..... 6
I. Mathematical Definition..... 6
II. Choosing ϵ (The Privacy Parameter)..... 7
III. Applications for LogLog Algorithm..... 9
2.5 Research Questions And Goals..... 11
3. Methodology..... 11
4. Results and Discussion.....12
4.1 Global Sensitivity and Distance of Worlds..... 12
4.2 Epsilon Selection.....14
4.3 Standard Error of Differentially Private Cardinality Estimation..... 15
5. Conclusions and Recommendations.....16
6. References.....17
Appendix A..... 18
Appendix B.....19
The Scaling Factor: Alpha..... 19
Merging Data Sketches..... 19

Abstract

Big data is essential for market, educational, and medical research to gain valuable insights. However, since 402 million terabytes of data are created per day, simply storing all of the data is not an efficient method to process it. This is because the data storage increases linearly to the dataset size. Also, when parties share data using de-identification techniques they do not protect individuals' data because of the possibility of combining other datasets to identify individuals (re-identification). This research aimed to tackle the issue of data re-identification for web-scale datasets in cardinality estimation using an **efficient, secure, and practical** algorithm. To estimate cardinality, the LogLog algorithm and its optimized version HyperLogLog, which used constant memory, were implemented. Then, the LogLog algorithm was modified by applying differentially private mechanisms, such as sampling elements and a unique element requirement, to mathematically guarantee privacy. Next, the global sensitivity and distance of worlds were calculated to find optimized epsilon values. The results showed optimal epsilon values of four (LogLog) and six (HyperLogLog). This allowed the calculation of the new algorithm's standard error. The modified differentially private returned negligible difference in standard error when compared to the unmodified LogLog and HyperLogLog algorithms. This result was significant because differential privacy was applied into practice using constant memory while still maintaining optimized accuracy for web-scale data operations.

1. Introduction

Data is essential to conduct market research, advance medical fields, and more. About 402 million terabytes of data are created per day (Duarte, 2023). However, at the same time **440** million records are exposed yearly in the United States (Petrosyan, 2023). These attacks mostly concern security breaches that can be protected with more secure encryption.

Another large source of data, such as public records and medical research, is publically available for use. These datasets often use de-identification techniques which remove user specific identities (name, social security number, date of birth). However, these techniques have been proven to not preserve privacy (Miller, 2021). This is possible because attackers can pair records in public datasets to draw information about specific individuals, a process known as re-identification.

Additionally, the volume of this data needs to be processed efficiently without the need of storing all items. In web-scale datasets, companies cannot store all the data because the volume is in order of petabytes. In this study, solutions for data privacy to prevent **re-identification** attacks with constant memory storage were explored. A web-scale solution to safely estimate cardinalities of datasets using differential privacy, a mathematically guaranteed mechanism to protect individuals' sensitive data in datasets, was implemented.

2. Background

2.1 Counting with The Power Law

Counting unique elements in a dataset is fundamental to data processing to draw useful statistics. A naive way to count would be to store all elements and count the items one by one, resulting in 100% accuracy. However, this uses a **linear** amount of memory, to the number of elements in the dataset, and thus becomes very expensive when the dataset size is of petabytes.

Another way to count was using the length of least significant (the far-most right) zeros of the binary representation of all the elements in the dataset. This followed the Exponential Distribution (the Power Law) (Nguyen, 2022). If there are many continuous zeros, it is a "rare" event, which translates into a dataset with a large size (Nguyen, 2022). In other words, long lengths of least significant zeros must come from a large sample size, which would be a large dataset. This method was applied to the LogLog algorithm and returned a final averaged estimate using **constant** memory.

2.2 LogLog Algorithm

This section built upon the intuition of the length of least significant zeros and provided an implementation of the LogLog algorithm which estimates cardinality of datasets. Consider a set of variables such that the rank of the current element, r, is the length of least significant zeros; M is the number of buckets (known as estimators); *k* is the number of bits to index/encode estimators, equal to $\log_2 M$. Also, *R*, is a list where the M buckets were stored.

Below was the pseudocode of the LogLog algorithm:

```
ALGORITHM LogLog (INPUT k: number of bits to encode estimators, D: domain of all
elements, alpha)

ASSUME M = 2**k
ASSUME alpha = 0.396

INITIALIZE a list of M registers, R

FOR item in D DO

     SET h = hash(item)
     SET binary_item = CALL convert_to_binary_representation(h)
     SET register_bits = binary_item [0:k]
     SET rank_bits = binary_item[k::]

     SET rank = CALL count_number_of_continuous_zeros(rank_bits)
     SET register_determined = CALL convert_to_number_from_binary(register_bits)

     IF R[register_determined] < rank THEN
          R[register_determined] = rank

RETURN  n̂ = α · M · 2^((1/m)·Σ_{j=0}^{M} R[j])
```

Figure 1. The LogLog Algorithm

Practically, data would be flowing through a stream and as they arrive one by one the item would be processed. At no time is a data element being stored: except in the list *R*. What is left in *R* is a data sketch of the dataset. This representative sample of the dataset allowed the algorithm to return accurate results.

2.3 HyperLogLog Algorithm

The LogLog algorithm can be improved by estimating the cardinality based on **only** the lowest **70%** of ranks (when sorted in ascending order) in *R*. This algorithm is called HyperLogLog. This limits the sensitivity of the algorithm because rare lengths of continuous zeros (high rank values) will not impact the final estimate. It is also more accurate because of its lower sensitivity. It was therefore possible to significantly reduce the standard error:

$$standard\,error = \frac{1.04}{\sqrt{M}} \tag{1}$$

With this, a new α value of 0.721 was necessary.

2.4 Differential Privacy

Differential privacy is a rigorous mathematical framework to bound the risk of revealing data about any individual in a dataset. This mathematical framework is guaranteed for any individual in the dataset if the query satisfies differential privacy. In the following sections, this study explored the definition, important terminology, how to select ϵ, and the applications for the LogLog algorithm.

I. Mathematical Definition

A query function, F, was ϵ-differentially private if the bound (2) was met for all possible datasets in world *W* (the possible elements in the dataset):

$$e^{-\epsilon} < \frac{P(F(D) = S)}{P(F(D\prime) = S)} < e^{\epsilon} \tag{2}$$

Where ϵ was the privacy budget. Additionally, D′ are all of the possible datasets which differ in a single element (where one dataset had an item removed) from D. In other words, differential privacy bounds the probability that an algorithm will **change** given the **absence** or **addition** of a data item. The goal of applying mechanisms to satisfy differential privacy is to limit the change of the algorithm's results while maintaining accuracy.

II. Choosing ϵ (The Privacy Parameter)

A barrier in differential privacy research is selecting an appropriate value of ϵ. This required the knowledge of the entire dataset and all possible datasets of D′. This was important since ϵ determines the probability of revealing information and the accuracy of the algorithm. A higher ϵ value indicates more accuracy but less privacy. Whereas a lower ϵ value indicates higher security but a lack of accuracy. In order to select ϵ, two concepts, global sensitivity and global maximum distance of worlds, were introduced.

First, ***global sensitivity*** (Δf) measures the maximum value that a query F (LogLog algorithm) changes when a **single** element from the dataset was removed for all possible removals.

$$\Delta f = max\,|F(D) - F(D\prime)| \tag{3}$$

The higher the global sensitivity, the more prone the query function was to attack. Ideally, the global sensitivity should be close to zero (where it is impossible to identify if an individual is in the dataset) for most query requests. With a traditional counting query, the global sensitivity is always one. However, the LogLog algorithm can vary since it estimates based on the length of continuous zeros.

Another variable that was calculated was the distance of the value of the query between the worlds (Δv). All the possible worlds of the original dataset are the possible datasets of D′. Δv is the maximum distance of F(D′) when **compared among each other** for all possible D′ datasets in the world *W*. To understand how these definitions are applied in practice, suppose the world *W* consisted of the values below where the letters are the users and the numbers corresponded to the user's bank account balance:

Table 1. Dataset of Users and Bank Account Balance

User ID	Balance
A	1
B	2
C	2
D	4

Our possible worlds consisted of all the possible datasets of size three (variations of removal):

Table 2. Possible Worlds in W and Data Items

World	Dataset
1	[2, 2, 4]
2	[1, 2, 4]
3	[1, 2, 4]
4	[1, 2, 2]

In the example, suppose the query function F represented the cardinality of the dataset.

Table 3. Cardinality Estimation of Worlds

World	F (World)
1	2
2	3
3	3
4	2

Recall that the global sensitivity is the maximum distance between the worlds and the original dataset which equals the following:

$$\Delta f = |F(W) - F(W_1)| = 3 - 2 = 1 \tag{4}$$

Δv, would be as below:

$$\Delta v = |F(W_1) - F(W_2)| = |2 - 3| = 1 \tag{5}$$

Being able to calculate the global sensitivity and maximum distance between worlds allowed an effective selection of ϵ. Δv, while exactly the same in the example, is not always the same. A separate experiment had to be performed.

Recall, differential privacy bounds the probability the attacker can positively or negatively identify an individual. The posterior probability of the attacker's belief of a correct guess is as bounded (Lee, 2011).

$$B(W_i) = \frac{1}{1 + (n-1) * e^{\frac{-\epsilon \Delta v}{\Delta f}}} \tag{6}$$

Where n is the size of the dataset. Rearranged for choosing ϵ to minimize the probability of the posterior belief (p):

$$\epsilon \leq \frac{\Delta f}{\Delta v} * ln[\frac{(n-1) * p}{1-p}] \tag{7}$$

It is important to note that Δf and Δv must be computed **before** any differentially private mechanisms were added. By calculating these values ϵ could be selected based on the desired value of p. This would allow the algorithm to be applied into practice. These bounds were often considered worst case situations since the maximum sensitivity may occur very infrequently.

III. Applications for LogLog Algorithm

As research has proven, the LogLog algorithm does not satisfy ϵ-differential privacy when **unmodified** (Desfontaines, 2018). However, there are some mechanisms and parameter guarantees that can be used to satisfy the privacy requirement. First, there must be at least n unique items in the data stream (Dickens, 2023).

$$n = \frac{M}{1 - e^{-\epsilon}} \tag{8}$$

Where M is the number of buckets. Second, before items are hashed into the registers, items must be chosen to be selected to hash or not. The probability of sampling these data elements was as follows (Dickens, 2023).

$$P(Item\ is\ Processed) = 1 - e^{-\epsilon} \tag{9}$$

To summarize, the updated algorithm process for differential privacy:

```
ALGORITHM Differentially Private LogLog (INPUT k: number of bits to encode estimators,
D: domain of all elements,ϵ: privacy budget)

ASSUME M = 2**k
ASSUME The number of unique elements in (8) is satisfied
ASSUME probability_sample = 1 - e**(-ϵ)

INITIALIZE a list of M registers, R

SET probability_hash = random.random()

IF probability_hash < probability_sample THEN

     FOR item in D DO
          SET h = MD5_hash(item)
          SET binary_item = CALL convert_to_binary_representation(h)
          SET Register_bits = binary_item [0:k]
          SET Rank_bits = binary_item[k::]

          SET rank = CALL count_number_of_continuous_zeros(rank_bits)
          SET register_determined = CALL convert_to_number_from_binary(register_bits)

          IF R[Register_determined] < rank THEN
               R[Register_determined] = rank

Return  n̂ = α · M · 2^((1/m)·Σ_{j=0}^{M} R[j])
```

Figure 2. Differentially Private LogLog Algorithm

2.5 Research Questions And Goals

The research aimed to answer two questions:

1. What mechanisms can be applied to allow LogLog to satisfy differential privacy when calculating cardinality estimation and sampling?
2. What value of ϵ provides optimal results for HyperLogLog and LogLog?

This research upgraded a version of LogLog and HyperLogLog (its optimized version) to satisfy ϵ-differential privacy to protect datasets that were vulnerable to a re-identification attack.

3. Methodology

For all experiments, the code was implemented in Python on a local Jupyter Notebook running on a Windows desktop. All datasets were self-generated uniformly random datasets using the Python random library and consisted of 64-bit numbers. It was not useful to test other distributions of data because they did not affect the algorithm's performance (Nguyen, 2022). These experiments involved testing the LogLog and HyperLogLog algorithm which estimate the cardinality of datasets. The value of k for the first and second experiment was 16. α for the LogLog algorithm was 0.396 while for the HyperLogLog algorithm it was 0.721. The hash function used was from the MD5 Python library. Larger datasets were used to increase accuracy and return more sensitive results. It was not necessary in this research to use open-source datasets because all items were hashed. 500 trials were completed for each experiment to ensure consistency. The results were plotted using MatPlotLib.

The first experiment compared the Δf as the dataset size increased. The dataset sizes were from 100,000 to 1,000,000 with a step of 50,000 in the experiments. For each dataset size, a new dataset was generated, D. Then, 500 (the number of trials) D′ datasets, that were one element different from D, were generated. Then, the cardinality of all of the D′ datasets were calculated using the LogLog algorithm and compared them to the estimated cardinality of D. The maximum distance for each dataset size and the number of sensitivities above zero were recorded.

The second experiment compared Δv as the dataset size increased. The dataset sizes were from 100,000 to 1,000,000 with a step of 50,000 in the experiments. For each dataset size, a new dataset was generated, D. Then, 500 (number of trials) datasets (D′) that were one element different from D were created. Then, the maximum distance between all of the D′ datasets was computed.

After completion the first and second experiments, the plot of the probability of the attacker's posterior belief (6) was generated and an ϵ value was chosen for a dataset of size one million. Values of ϵ between 0 and 10

with a step of 0.01 were chosen. This allowed a selection of ϵ. The highest ϵ value was selected for both algorithms where the value of (6) was 0.0025.

Finally, the standard error of the differentially private LogLog and HyperLogLog and the original algorithms were plotted to see how the accuracy was affected when compared with the size of k. The standard error of the algorithm was computed with a dataset of size one million. Values of k between 10 and 23 were used. Twenty trials of the experiment were run which utilized maximum computational power.

4. Results and Discussion

4.1 Global Sensitivity and Distance of Worlds

The global sensitivity of the LogLog and HyperLogLog algorithm with increasing dataset sizes was shown in Figure 3.

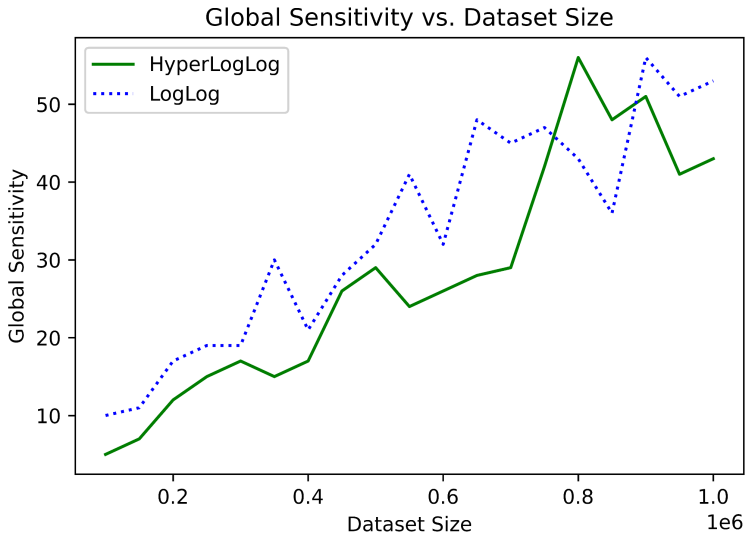


Figure 3. Global Sensitivity vs. Dataset Size

It can be observed that both lines generally follow an increasing **linear** relationship. This was expected since as dataset sizes increase it is more likely to observe a large number of continuous zeros. A large number of continuous zeros would incur a larger estimate which could lead to an increase in sensitivity. In other words, as the algorithm increases accuracy, as the datasets get larger, it leads to more sensitivity if a rare length of continuous zeros is removed.

HyperLogLog was almost always less sensitive than the LogLog algorithm. This was evident since HyperLogLog only uses the lower 70% of ranks. This means rare events (lengths of continuous zeros) do not influence the sensitivity leading to less sensitivity overall. The LogLog algorithm accounts for all registers for

the final estimate which means the sensitivity will almost always be above zero. The number of sensitivities above zero of the LogLog and HyperLogLog algorithm was shown in Figure 4.

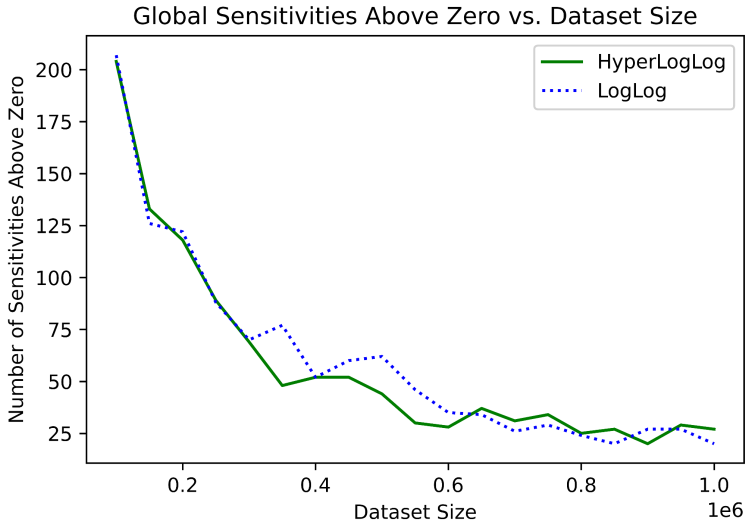


Figure 4. Sensitivities Above Zero vs. Dataset Size

In this figure, it was shown that the number of sensitivities above zero exponentially **decayed** as the dataset size increased. In fact, the data suggested that the mean of these sensitivities was below one (the sensitivity of traditional counting). So, the global sensitivity was an **absolute worst** case situation that does not reflect most of the sensitivities that occur. At a dataset of size 100,000, about 40% of the worlds had a sensitivity above zero but at one million the percent of sensitivities above zero dropped to 5%. Next, the global distance of worlds of the LogLog and HyperLogLog algorithm was shown in Figure 5.

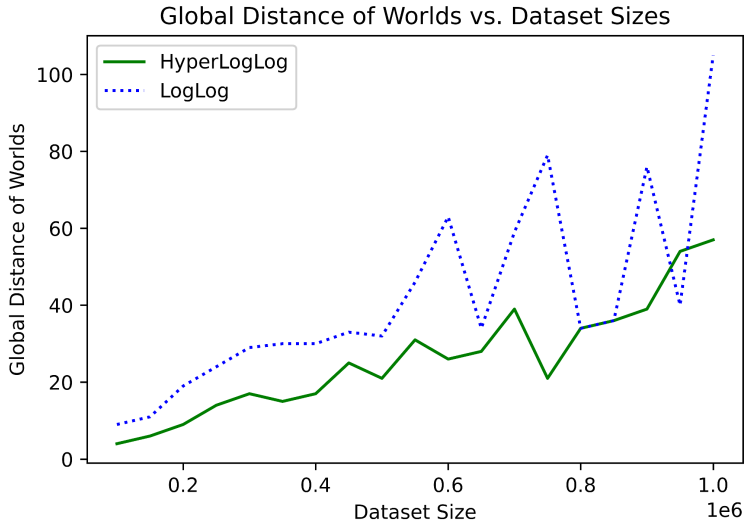


Figure 5. Global Distance of Worlds (Δv) vs. Dataset Size

The HyperLogLog algorithm is almost always below the global distance of worlds of the LogLog algorithm. Additionally, the green line (HyperLogLog) resembled a more positive linear correlation than the blue line (LogLog). These global distances were very extreme situations as most of the estimates are very close to the real dataset size. It is evident that HyperLogLog has a lower global distance of worlds than LogLog.

4.2 Epsilon Selection

The values of ϵ and upper bound probability (6) for the LogLog and HyperLogLog algorithm were shown in Figure 6.

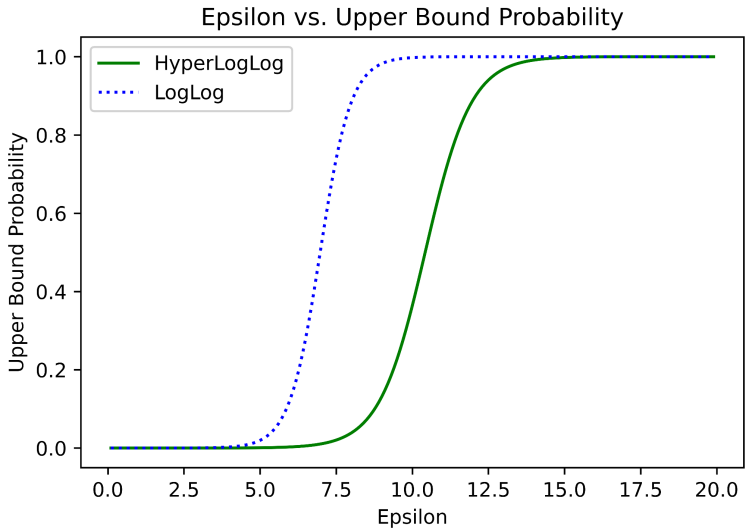


Figure 6. Epsilon vs. Upper Bound Probability Comparison

The functions in Figure 6 followed a **logistic** relationship. Recall that the goal is to maximize ϵ to gain the most accurate result of the analysis while still minimizing the probability. A logistic function presents an advantage since there was a section where the function was mostly horizontal, where the probability stays relatively constant, for increasing epsilons until it rapidly increased. This figure proves the HyperLogLog algorithm was superior to the LogLog algorithm in differential privacy because it remains a **lower** probability for more ϵ values. This makes sense because of the lower sensitivity of the algorithm compared to the LogLog algorithm.

Consider the more zoomed up plot (Figure 7) which has ϵ values between zero and seven.

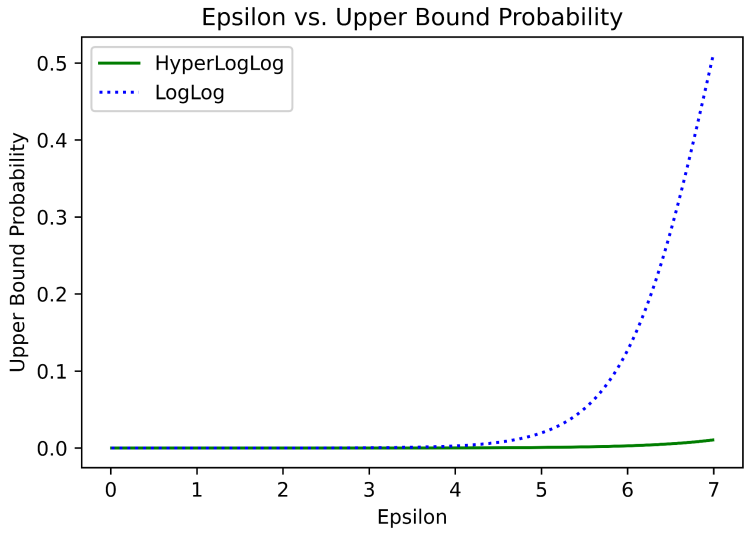


Figure 7. Epsilon vs. Upper Bound Probability Comparison Zoomed Up

Here the clear advantage was visible for HyperLogLog since as the accuracy increases (when ϵ increases) there is not a significant increase in the upper bound probability. A value of four (LogLog algorithm) and six (HyperLogLog algorithm) for ϵ were selected for optimal calculations.

4.3 Standard Error of Differentially Private Cardinality Estimation

The standard error of the LogLog algorithm, its differentially private version, and theoretical error were shown in Figure 8 with a ϵ value of four.

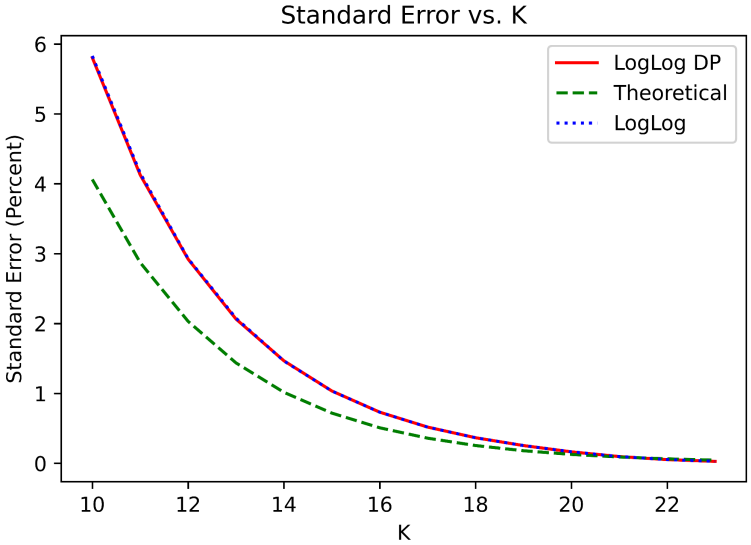


Figure 8. Standard Error vs. K: LogLog Algorithm

Through this figure, it was concluded there was a negligible difference in standard error between the LogLog DP (Differentially Private) and LogLog algorithm. The Mean Absolute Error was 1.446 for LogLog DP and 1.357 for LogLog when compared to the theoretical standard error. This proved that small changes to the algorithm can mathematically protect this data while compromising negligible accuracy for the estimation. This was significant since this showed the algorithm initially was nearly differentially private.

The standard error of the HyperLogLog algorithm, its differentially private version, and theoretical error were shown in Figure 9 with a ϵ value of six.

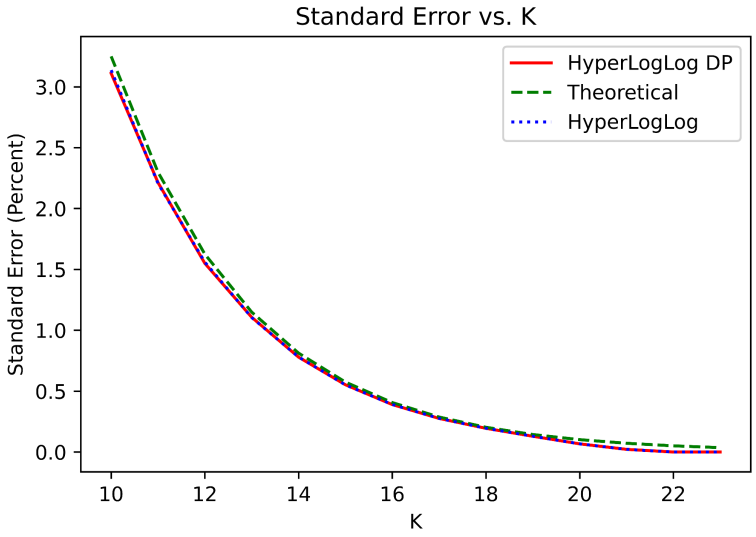


Figure 9. Standard Error vs. K: HyperLogLog Algorithm

Again, there is a negligible difference in accuracy between the HyperLogLog algorithm and the new differentially private version. The Mean Absolute Error of HyperLogLog DP was 0.122 and HyperLogLog was 0.131 when compared to the theoretical error. This was significant because it shows no sacrifice in security for privacy of the dataset. There was a significant increase in accuracy of the algorithm compared to LogLog while still limiting the probability of an attack.

5. Conclusions and Recommendations

In conclusion, data privacy is an important issue as more data is available. As more publicly available data is available for hackers to use it will be possible for them to re-identify individuals. The issue of processing web-scale datasets efficiently with constant memory was tackled. Differentially private mechanisms were added

to the LogLog and HyperLogLog algorithm to mathematically guarantee privacy. Then, the upper bound probability (6) was used to determine appropriate values for ϵ. This resulted in ϵ values of four (LogLog algorithm) and six (HyperLogLog algorithm). When the standard error of the differentially private algorithms was tested, a **negligible** difference in standard error was observed. This was a groundbreaking result because it showed accuracy would not be sacrificed for security. As predicted, the HyperLogLog algorithm was superior in the areas of global sensitivity, global distance of worlds, and standard error. While many research papers have failed to put into practice differential privacy, this study developed practical ϵ values to implement a differentially private algorithm for secure web-scale cardinality estimation.

An important recommendation is that future research should strive to find a more efficient way to select ϵ to put differential privacy into practice. It required lots of computational power and often cannot be enumerated for all possibilities. Without ϵ values, differential privacy cannot be implemented. Also, implementing differential privacy for researchers and industries is a top priority because of the high risk of re-identification.

6. References

1. Allen, M. Health insurers are vacuuming up details about you—And it could raise your rates. ProPublica. https://www.propublica.org/article/health-insurers-are-vacuuming-up-details-about-you-and-it-could-raise-your-rates
2. Amount of data created daily. (2023, March 16). Exploding Topics. https://explodingtopics.com/blog/data-generated-per-day
3. Desfontaines, D., Lochbihler, A., & Basin, D. (2018). Cardinality estimators do not preserve privacy (arXiv:1808.05879). arXiv. https://doi.org/10.48550/arXiv.1808.05879
4. Dickens, C., Thaler, J., & Ting, D. (2023, February 3). Order-invariant cardinality estimators are differentially private. https://par.nsf.gov/servlets/purl/10417620
5. Hehir, J., Ting, D., & Cormode, G. (2023). Sketch-flip-merge: Mergeable sketches for private distinct counting. Proceedings of the 40th International Conference on Machine Learning, 202, 12846–12865.
6. Lee, J., & Clifton, C. (2011). How much is enough? Choosing ε for differential privacy. In X. Lai, J. Zhou, & H. Li (Eds.), Information Security (pp. 325–340). Springer. https://doi.org/10.1007/978-3-642-24861-0_22
7. Near, J., & Abuah, C. (n.d.). Programming differential privacy.
8. Miller, K. (2021, July 19). De-identifying medical patient data doesn't protect our privacy. Stanford University. https://hai.stanford.edu/news/de-identifying-medical-patient-data-doesnt-protect-our-privacy
9. Nguyen, T. (2023). Sublinear Counting With The Power Law. Retrieved January 15, 2024. Washington State Science and Engineering Fair 2023.
10. Nguyen, T. , (2022, March 4). The Power Law of Continuous Zeros. Retrieved December 30, 2023. Washington State Science and Engineering Fair 2022.
11. Nissim, K., Steinke, T., Wood, A., & Bun, M. (2017). Differential privacy: A primer for a non-technical audience. https://privacytools.seas.harvard.edu/files/privacytools/files/pedagogical-document-dp_0.pdf
12. *Number of data breaches and victims U.S. 2022*. (n.d.). Statista. Retrieved January 21, 2024, from https://www.statista.com/statistics/273550/data-breaches-recorded-in-the-united-states-by-number-of-breaches-and-records-exposed/

Appendix A

Example of Re-identification attack

Simply defined, a re-identification attack is the process of matching anonymized data (that is de-identified) with public records or other datasets to determine if a specific individual belongs or does not belong in a dataset. As stated in the paper, these attacks are subtle since they often don't require the need of passing firewalls or ciphers, they just involve pairing records. With powerful computers and AI today, these attacks are a threat since computers can easily compute these tasks.

To illustrate re-identification attacks, let us pretend Alice and Bob are researchers at Medical University and completed an analysis of a 100-person clinical trial for Vaccine X and intend to publish the results of their analysis publically. An attacker, Eve, looked to gain valuable insight from this analysis. On Monday, Alice released de-identified data that stated 24% of participants had a disability and were over 40. However, on Tuesday, Bob updated the de-identified data to reflect 25% of participants had a disability and were over 40. Between these two data releases, participant Joe joined the trial, which prompted the update of the data. Within hours before Bob updated the results, Joe posted on his Instagram and Facebook about his participation in Vaccine X and the contribution he made.

As Eve scanned the post she noticed a discrepancy. Why did the data vary between 24% and 25%? She concluded that somebody of age 40 must have completed the vaccine dose between Monday and Tuesday and had a disability which led to the update of the data. This was because if there were 100 people in the trial, 1% represented one person. As she dug across Instagram and Facebook to identify the individual, she came across Joe's post. It stated, "I participated today in the Vaccine X and I am proud of advancing science." Eve began to smile as she confidently confirmed it was Joe.

It also happens to be that Eve was an employee at Quick Insurance LLC and she used this data to raise Joe's rate of health insurance because of his higher medical risks.

Joe received the message and was shocked why the insurance company knew about his recent disability and called them to ask if they violated any privacy acts. They replied that they used only open source records to make the decision to raise his rates.

These situations are becoming increasingly common as insurance companies gain more information about purchases and activities that could lead to higher insurance rates (Allen, 2018). Simple actions such as buying plus size clothing, changing a legal name could lead to raises of insurance (Allen, 2018).

This was why differential privacy was used. If the results of any data releases had a specific amount of noise (i.e. adding a differentially private mechanism) it would be impossible for Eve to know the demographics of Joe even with the post. For example, if Bob released a data analysis that said 24% were above 40 and had a disability, Eve could not conclude if Joe was disabled. These attacks have implications in social and medical research and big data. Without mathematically bounding the risk, it is impossible to know if the anonymized data is safe from re-identification.

Appendix B

Properties of the LogLog Algorithm

The Scaling Factor: Alpha

Calculating the cardinality of a dataset is important when methods are estimated. In order to know if the LogLog algorithm needed scaling, the relationship between real and estimated cardinality was plotted. Since the estimation possessed a linear relationship with the actual dataset size, it was possible to multiply the estimate by a value α to correct the algorithm close to the actual cardinality of the dataset.

Merging Data Sketches

Another property of the LogLog algorithm was the ability to merge data sketches or ranks, *R*. This can be simply done by compiling the maximum ranks at each estimator (across the sketches) and then computing the final estimation. This was a major advantage. For example, consider if Company A wants to compile data with Company B and C to see the trends in their industry. If each company kept their own data sketch, they can easily combine their sketches to return an estimate. Even if all of the raw data is gone it still contains highly useful information. It even accurately samples data items which can be used to calculate sizes of sub-populations (Nguyen 2023). The one limitation is that it must be done with the same value of *k*. There are no compromises of accuracy as a result of this operation as well.